

1 概述

本次信息检索的实验是本次作业针对南开校内资源构建 Web 搜索引擎，为用户提供南开信息的查询服务和个性化推荐服务。本次实验收集了南开大学新闻网、校史网以及各学院官网的数据，构建了一个能够搜索和查询南开校内各种信息的综合性 web 搜索引擎。本实验主要基于 python 和 flask 框架实现，程序设计主要分为以下几个部分：

- 网页抓取
- 文本索引
- 查询服务
- 个性化查询
- web 页面
- 个性化推荐

页面的整体设计如下图：



2 设计实现

2.1 网页爬取

在本次实验中我们要实现校内信息的搜索引擎，首先我们利用爬虫程序爬取了包括南开新闻网、南开校史网以及南开大学各学院官网等二十多个网站的网页信息，并将网页信息保存在 json 文件夹中。

我们的爬虫的基本功能是从指定的网址出发，爬取页面的所有链接，然后再由获得的链接不断进行爬取，同时将每个网页的基本信息保存在指定的 json 文件中，将解析的 html 源码单独保存，用于后续完成网页块中功能。具体的实现如下：通过读取 config.json 中的起始 URL、白名单域名、文档后缀及排除后缀等参数，自动构建抓取任务；在主程序中使用 ThreadPoolExecutor 为每个站点启动并发线程，大幅提升抓取效率。每条抓取流程中，它首先发送 HTTP 请求并自动检测页面编码，然后借助 BeautifulSoup 解析 HTML——凡是指向 PDF、Word 等文档的链接，立即提取标题和 URL 存入结果列表；对白名单域名内的普通页面链接，进行队列管理，实现递归抓取。为保证稳定性，爬虫会每隔一定条数（batch_size）将内存中的结果批量写入 JSON 文件，并定时（state_interval）将已访问

URL 和待爬队列持久化到 `state/` 目录，这样即便程序意外中断，也能从上次断点迅速恢复。它还会为每个成功抓取的 HTML 页面生成本地快照（保存在 `html_snapshots/`），同时提取页面 `<title>` 和纯文本内容以便后续检索和分析。全局计数器在多线程环境下通过锁保护，累计所有站点的抓取量，并在达到预设上限（`max_total`）时自动触发停止事件，防止“爬虫失控”。

爬虫的伪代码如下：

Algorithm 1 简化并发断点续爬爬虫伪代码

Input: `config.json`, 起始 URL 列表 `sites`

Output: 抓取结果保存在 `json/`，页面快照保存在 `html_snapshots/`

```

1: function GET_PAGE(url)
2:   发送 HTTP 请求，检测并设置正确编码
3:   return 响应对象 resp
4: end function
5: function SAVE_STATE(site_name, visited, queue)
6:   将 visited 与 queue 写入 state/
7: end function
8: function CRAWL_SITE(start_url, whitelist)
9:   初始化 results, visited, queue
10:  while queue 非空且未触发停止事件 do
11:    url  $\leftarrow$  queue.pop(0)
12:    resp  $\leftarrow$  GET_PAGE(url)
13:    if resp.status_code == 200 then
14:      解析 HTML，遍历所有链接
15:      文档链接  $\rightarrow$  直接存储到 results
16:      普通页面链接且域在 whitelist  $\rightarrow$  入队
17:      保存页面快照，提取 title 和 content 加入 results
18:    end if
19:    更新全局计数，若超限则触发停止事件
20:    定期批量写入 results 到 JSON
21:    定期调用 SAVE_STATE(...)
22:    暂停 delay 秒
23:  end while
24:  将剩余 results 写入 JSON，清理状态文件
25: end function
26: 主程序:
27: 并发启动 CRAWL_SITE(start_url, whitelist), 线程数至多 max_workers

```

每个网站官网的数据分别保存在各自 `json` 文件中，如下图所示。每个网页的 `json` 数据保存着 `source`, `type`, `url`, `title`, `content`, `snapshot` 数据。其中数据被划分为两类，其中一类是“html”，表示普通的网页，“document”表示是 pdf/doc 文件类数据，用于后续的文档查询功能的实现。

```
{
  "source": "chem_nankai_edu_cn",
  "type": "html",
  "url": "https://chem.nankai.edu.cn/",
  "title": "南开大学化学学院",
  "content": "南开大学化学学院 EN 学院概况 学院简介 发展历程 学院领导 系所单位 办公室 学科简介 大师风采 师资力量 师资队伍",
  "snapshot": "html_snapshots\\chem_nankai_edu_cn\\index.html"
},
```

经过简单的数据清洗和数据处理后，我们统计所有 json 数据总量，结果显示，我们总共爬取了二十多个南开校内网站的网页数据，总量达到了 108275 条数据。

```
D:\web搜索引擎\venv\Scripts\python.exe D:\web搜索引擎\处理\countnumber.py
ai_nankai_edu_cn.json: 5066 条记录
bs_nankai_edu_cn_1_new.json: 9673 条记录
bs_nankai_edu_cn_2_new.json: 8682 条记录
bs_nankai_edu_cn_3_new.json: 2433 条记录
ceo_nankai_edu_cn.json: 2679 条记录
chem_nankai_edu_cn.json: 2014 条记录
cs_nankai_edu_cn.json: 2227 条记录
cz_nankai_edu_cn.json: 3979 条记录
economics_nankai_edu_cn.json: 2515 条记录
env_nankai_edu_cn.json: 6429 条记录
finance_nankai_edu_cn.json: 7129 条记录
history_nankai_edu_cn.json: 2106 条记录
jc_nankai_edu_cn.json: 1997 条记录
law_nankai_edu_cn.json: 2140 条记录
math_nankai_edu_cn.json: 2430 条记录
medical_nankai_edu_cn.json: 2484 条记录
news_nankai_edu_cn.json: 4853 条记录
physics_nankai_edu_cn.json: 9839 条记录
sfs_nankai_edu_cn.json: 14682 条记录
sky_nankai_edu_cn.json: 475 条记录
tas_nankai_edu_cn.json: 8188 条记录
wxy_nankai_edu_cn.json: 2427 条记录
xs_nankai_edu_cn.json: 334 条记录
zfxxy_nankai_edu_cn.json: 3494 条记录
=====
所有 JSON 文件共计 108275 条记录
```

2.2 文本索引

在这部分中，我们将以用倒排索引实现文本的索引。首先，遍历 json 目录下的所有记录文件，为每条文档分配一个唯一的 doc_id，提取标题 (title)、URL、内容摘要 (snippet)、类型和快照路径等元信息，保存在 doc_meta.json 中，然后将标题和正文拼成一个字符串，先用正则把英文、数字和连续中文片段拆分，再对中文部分用 Jieba 细分，同时过滤停用词，生成一个按词项 (term) 索引到文档和词位列表 (positions) 的倒排索引，最终写入 inverted_index.json 中，构成完整的倒排索引。通过这种方式，既支持中英文混合检索，也能快速定位每个词在文档中的出现位置。

实现的主要代码如下：

```
1 def build_index():
2     os.makedirs(OUTPUT_DIR, exist_ok=True)
3
4     inverted_index = defaultdict(lambda: defaultdict(list))
5     doc_meta = {}
6
7     next_doc_id = 1
8
9     # 遍历 JSON_DIR 下所有 .json 文件
10    for fn in os.listdir(JSON_DIR):
11        if not fn.lower().endswith(".json"):
12            continue
13        full_path = os.path.join(JSON_DIR, fn)
14        with open(full_path, "r", encoding="utf-8") as f:
15            try:
16                records = json.load(f)
17            except Exception as e:
18                print(f"跳过无法解析的文件 {fn}: {e}")
19                continue
20
21        for rec in records:
22            rec_type = rec.get("type", "html")
23            title = rec.get("title", "").strip()
24            url = rec.get("url", "").strip()
25            snippet = ""
26
27            # 区分 html vs document
28            if rec_type == "html":
29                content = rec.get("content", "").strip()
30                snippet = content[:100] + ("..." if len(content) > 100 else "")
31                raw_snap = rec.get("snapshot", "").strip()
32            else:
33                # document 类型只用 title 作为可检索内容
34                content = title
35                snippet = title[:100] + ("..." if len(title) > 100 else "")
36                raw_snap = "" # 文档一般没有快照
37
38            # 分配 doc_id
39            doc_id = str(next_doc_id)
```

```

40         next_doc_id += 1
41
42     # 存储文档元信息
43     doc_meta[doc_id] = { "title": title, "url": url, "snippet":
44                           snippet, "type": rec_type, "snapshot": snap
45     }
46
47     # 将 title + content 拼成一个大字符串，分词并记录位置
48     full_text = f"{title} {content}"
49     tokens_list = tokenize(full_text)
50     for pos, term in enumerate(tokens_list):
51         inverted_index[term][doc_id].append(pos)
52
53 # 序列化写入磁盘 —— 将 postings 转成 [(doc_id, [pos_list]), ...] 形式
54 serializable_index = {}
55 for term, posting_dict in inverted_index.items():
56     postings = []
57     for doc_id, pos_list in posting_dict.items():
58         postings.append((doc_id, pos_list))
59     serializable_index[term] = postings
60
61 with open(INVERTED_INDEX_FILE, "w", encoding="utf-8") as f:
62     json.dump(serializable_index, f, ensure_ascii=False, indent=2)

```

我们打开对应的索引文件，发现已经成功实现了倒排索引的功能。

```

[["106605", [133]], ["106606", [226]], ["106609", [1336]], ["106617", [472]], "代宇嘉": [{"105276", [1022]}, ["106193", [163]], ["106609", [302]], "玉叶":
[["105276", [1043]], ["106193", [162]], ["106592", [398]], ["106609", [975]], "杜恒易": [{"105276", [1051]], "罗佳瑞": [{"105276", [1058]], ["106609", [509]],
"2120232653": [{"105276", [1094]], "马茜": [{"105276", [1104]], "吴昱昕": [{"105276", [1110]], "童瑶": [{"105276", [1112]}, ["105548", [136]], "杨紫婷":
[["105276", [1124]], ["106609", [885]], "王妍颖": [{"105276", [1125]], "高琦琦": [{"105276", [1137]], ["106609", [1266]], "高梦瑶": [{"105276", [1149]],
["105408", [576]], ["106448", [356, 636]], ["106502", [1294]], "2120222640": [{"105276", [1150]], "布姆": [{"105276", [1154]], "袁涵": [{"105276", [1161]],
["106591", [439]], "董泽琛": [{"105276", [1163]], ["106609", [1292]], ["106617", [454]], "邓力夫": [{"105276", [1167]], ["106609", [1179]], "苏娅宁":
[["105276", [1173]], "增白": [{"105276", [1177]], "刘晓奕": [{"105276", [1182]}, ["107842", [648]], ["108223", [935]], "温景怡": [{"105276", [1183]],
["106609", [851]], ["107837", [97]], ["107840", [123]], "肖润": [{"105276", [1190]], "棉": [{"105276", [1191]], ["106502", [1553]], ["107079", [471]], "马晶媛":
[["105276", [1219]], ["106609", [1412]], "2120232651": [{"105276", [1233]], "代涛": [{"105276", [1250]}, ["106596", [697]], ["106597", [287]], ["106605",
[582]], ["106606", [558]], ["106609", [566]], ["106617", [784]], ["107837", [128]], ["107840", [149]], "2012745": [{"105276", [1396]], "此终": [{"105276",

```

2.3 查询服务

为实现了更加精准的查询服务，我们需要实现向量空间模型，对查询结果进行排序。为用户提供站内查询、文档查询、短语查询、通配查询、查询日志、网页快照等共计六种高级搜索功能。

2.3.1 向量空间模型

向量空间模型（Vector Space Model, VSM）是信息检索中最经典、最常用的文档表示与相似度计算方法之一。其核心思想与实现要点包括：文档与查询的向量化表示、相似度度量。

首先是文本与查询的向量化表示：

- 维度（term）：将整个语料库中出现的所有词项（term）当作维度，每个文档和查询都被映射到这个高维空间里。

- 权重：常用的权重方案是 TF-IDF（词频-逆文档频率），即

$$\text{tfidf}_{t,d} = \text{tf}_{t,d} \times \log\left(\frac{N}{\text{df}_t}\right)$$

其中 $\text{tf}_{t,d}$ 是词 t 在文档 d 中出现的次数， df_t 是包含 t 的文档数， N 是文档总数。

然后是相似度计算，我们计算余弦相似度：

- 文档向量 d 与查询向量 q 的相似度通常用余弦值来衡量：

$$\text{sim}(\mathbf{d}, \mathbf{q}) = \frac{\mathbf{d} \cdot \mathbf{q}}{\|\mathbf{d}\| \|\mathbf{q}\|} = \frac{\sum_{i=1}^n d_i q_i}{\sqrt{\sum_{i=1}^n d_i^2} \sqrt{\sum_{i=1}^n q_i^2}}$$

- 余弦值越大表示方向越相近，文档与查询的相关性越高

在我们的代码中通过 `search_vsm` 函数实现了一个基于向量空间模型的全文检索打分器：它首先对用户查询进行分词并计算查询端的 TF-IDF 权重向量，然后遍历倒排索引中每个词的 `posting` 列表，累加文档端的 TF-IDF 点积，得到原始分数；接着用查询向量和文档向量的范数做余弦归一化，最后可选地对包含用户偏好关键词的文档加分，并按得分排序返回最高的前 K 个文档 ID。整个过程利用了预先构建好的 `inverted_index`（词 $\rightarrow$ (文档 ID, 词位列表)）、文档频次 `df`、总文档数 N 以及每篇文档的向量范数字典 `doc_norm`，在中英文混合场景下提供了高效、可扩展的检索能力。

实现的主要代码如下：

```

1 def search_vsm(query, candidate_docs=None, top_k=50, boost_terms=None,
2   boost_score=1.0):
3     terms = tokenize(query)
4     if not terms:
5         return []
6
7     tf_q = {}
8     for t in terms:
9         tf_q[t] = tf_q.get(t, 0) + 1
10
11     tfidf_q = {}
12     for t, fq in tf_q.items():
13         if t not in df:
14             continue
15         idf_t = math.log(N / df[t]) if df[t] > 0 else 0.0
16         tfidf_q[t] = fq * idf_t
17
18     norm_q = math.sqrt(sum(w * w for w in tfidf_q.values()))
19     score = {}
20
21     for t, w_q in tfidf_q.items():
22         postings = inverted_index.get(t, [])
23         idf_t = math.log(N / df[t]) if df[t] > 0 else 0.0

```

```

23     for doc_id, pos_list in postings:
24         if candidate_docs is not None and doc_id not in candidate_docs:
25             continue
26         tf_d = len(pos_list)
27         w_d = tf_d * idf_t
28         score[doc_id] = score.get(doc_id, 0.0) + w_q * w_d
29
30     # 余弦归一化
31     for doc_id in list(score.keys()):
32         if doc_id not in doc_norm or norm_q == 0:
33             score[doc_id] = 0.0
34         else:
35             score[doc_id] /= (norm_q * doc_norm[doc_id])
36
37     # 如果有 boost_terms (用户偏好关键词), 对包含这些词的文档加额外分数
38     if boost_terms:
39         for doc_id in list(score.keys()):
40             content_to_check = (doc_meta[doc_id]["title"] + " " +
41                                 doc_meta[doc_id]["snippet"]).lower()
42             for bt in boost_terms:
43                 if bt.lower() in content_to_check:
44                     score[doc_id] += boost_score
45                     break
46
47     ranked = sorted(score.items(), key=lambda x: x[1], reverse=True)
48     return [doc_id for doc_id, _ in ranked[:top_k]]

```

2.3.2 站内查询

现在, 我们实现基本的搜索引擎站内查询的功能。基本的站内查询很简单, 我们可以直接利用向量空间模型来实现站内查询。站内查询的基本功能是用户输入想要查询的语句, 然后页面返回对应的与之相关的文档。在 /search 路由中, 当用户提交普通查询 (不以 type:document 为前缀) 时, 后端会将查询字符串传入 search_vsm, 在全库范围内基于 TF-IDF 与余弦相似度计算前 50 条相关文档, 并利用用户偏好关键词对匹配文档进行加分; 随后再调用 reorder_by_click, 将用户历史点击过的文档优先展示, 最终从 doc_meta 中提取标题、URL、摘要和快照路径, 封装成结果列表送入模板渲染, 完成一次端到端的检索与个性化排序流程。

实现的具体代码如下:

```

1     if raw_q:
2         doc_prefix = "type:document "
3         if raw_q.startswith(doc_prefix):
4             actual_q = raw_q[len(doc_prefix):].strip()
5             candidate_docs = {doc_id for doc_id, meta in doc_meta.items() if
6                               meta.get("type") == "document"}
7             matched = search_vsm(actual_q, candidate_docs,
8                                  boost_terms=user_prefs, boost_score=1.0)
9         else:

```

```

8         matched = search_vsm(raw_q, None, boost_terms=user_prefs,
9                                boost_score=1.0)
10
11     matched = reorder_by_click(user_id, matched)
12     doc_ids_to_show = matched
13
14     for doc_id in doc_ids_to_show:
15         meta = doc_meta.get(doc_id, {})
16         results.append({
17             "doc_id": doc_id,
18             "title": meta.get("title", ""),
19             "url": meta.get("url", ""),
20             "snippet": meta.get("snippet", ""),
21             "type": meta.get("type", "html"),
22             "snapshot": meta.get("snapshot", "")
23         })
24
25     return render_template("search.html",
26                           query=query,
27                           results=results,
28                           page=page,
29                           total_pages=total_pages,
30                           username=username)

```

功能演示如下，在搜索引擎中选择“普通检索”，然后在搜索框中输入“信息技术”作为查询语句，然后点击搜索，页面会返回对应的文档和网页，每个网页会显示标题和简要的文本信息，我们点击网页的搜索结果，就会跳转到指定的页面。





2.3.3 文档查询

文档查询可以使用户通过关键词搜索到相应的文档，而不是网页，用户点击文件，将会直接打开对应的文件。而文档查询是通过在用户输入前加特殊前缀来触发的，在我们的程序中当用户输入查询文档的关键词 query 之后，前端代码将会自动将 "type:document" 作为前缀添加到 query 之前，然后传递给后端，后端先检查查询字符串是否以 "type:document" 开头，如果是，就去掉这个前缀得到真正的检索词 actual_q；接着，从事先加载的 doc_meta（文档元数据）里筛选出所有 type == "document" 的文档 ID，组成一个 candidate_docs 集合；最后将 actual_q 和这个文档 ID 子集一起传入 search_vsm，仅在文档中基于 TF-IDF + 余弦相似度进行打分并返回排名前 50 的结果，从而实现了“只在文档类型内容里搜索”的功能。

```

1 # 在 /search 路由中，检测 "type:document" 前缀后只检索文档
2 doc_prefix = "type:document "
3 if raw_q.startswith(doc_prefix):
4     actual_q = raw_q[len(doc_prefix):].strip()
5     # 只在 metadata 中 type=document 的文档集合里搜索
6     candidate_docs = {
7         doc_id
8         for doc_id, meta in doc_meta.items()
9         if meta.get("type") == "document"
10    }
11    matched = search_vsm(actual_q,
12                          candidate_docs=candidate_docs,
13                          top_k=50,
14                          boost_terms=user_prefs,

```

15

```
boost_score=1.0)
```

下面演示功能，选择“文档查询”，输入“考试报名”，然后点击搜索，出现下面的页面，我发现返回的结果全是相应的文档。



我们点击第一个结果，直接打开对应的 pdf 文件。



2.3.4 短语查询

短语查询实现了精确短语检索：它先把输入短语按中英文混合规则分词，然后在倒排索引里依次筛选出同时包含所有词且在文档中相邻出现、顺序一致的文档 ID。具体做法是，先取出短语第一个词的所有 posting（文档 → 位置列表），然后对短语的每个后续词，用新的 posting 与上一轮的候选集合和位置列表比对——仅保留那些在同一个文档中，且当前词的某个位置恰好等于前一个词某个位置加一的文档，逐步缩小候选集。最后返回所有完成所有词位置连续匹配的文档列表。

```

1 def phrase_query(phrase, candidate_docs=None):
2     terms = tokenize(phrase)
3     if not terms:
4         return []
5
6     postings0 = dict(inverted_index.get(terms[0], []))
7     if candidate_docs is not None:
8         postings0 = {doc: pos_list for doc, pos_list in postings0.items() if doc in
9                        candidate_docs}
10
11    if len(terms) == 1:
12        return list(postings0.keys())
13
14    candidate = set(postings0.keys())
15    for i in range(1, len(terms)):
16        term = terms[i]
17        postings_i = dict(inverted_index.get(term, []))
18        if candidate_docs is not None:
19            postings_i = {doc: positions for doc, positions in postings_i.items() if
20                           doc in candidate_docs}
21        new_cand = set()
22        for doc_id in candidate:
23            if doc_id not in postings_i:
24                continue
25            pos_prev = postings0.get(doc_id, [])
26            pos_curr = postings_i.get(doc_id, [])
27            if any(p + 1 in pos_curr for p in pos_prev):
28                new_cand.add(doc_id)
29        postings0 = {doc: postings_i[doc] for doc in new_cand if doc in postings_i}
30        candidate = new_cand
31        if not candidate:
32            break
33    return list(candidate)

```

下面是功能演示，我们选择“短语查询”功能，然后输入“信息检索”，返回带有完整关键词的网页。



2.3.5 通配查询

wildcard_query 函数实现了带有“*（多字符）”和“?（单字符）”通配符的词项检索：它首先把用户输入的模式（如“data*base?”）转成完整的正则表达式，然后遍历倒排索引中所有词项，对每个匹配该正则的 term，把它出现的文档 ID 和词频累加到一个 matched_docs 字典中；如果提供了 candidate_docs，还会在累加前过滤掉不在该集合的文档；最后根据累加的匹配频次降序排序，返回最相关的文档 ID 列表。

```

1 def wildcard_query(pattern, candidate_docs=None):
2     regex_pattern = "^" + re.escape(pattern).replace(r"\*", ".*").replace(r"\?", ".")
3     + "$"
4     regex = re.compile(regex_pattern)
5
6     matched_docs = {}
7     for term, postings in inverted_index.items():
8         if regex.match(term):
9             for doc_id, pos_list in postings:
10                 if candidate_docs is not None and doc_id not in candidate_docs:
11                     continue
12                 matched_docs[doc_id] = matched_docs.get(doc_id, 0) + len(pos_list)
13
14     ranked = sorted(matched_docs.items(), key=lambda x: x[1], reverse=True)
15     return [doc_id for doc_id, _ in ranked]

```

演示如下：输入“温 *”，返回含有以“温”开头的网页和文档。



2.3.6 查询日志

这段代码实现了查询日志与历史查询回显功能：

记录查询 (log_query): 每当用户提交搜索请求时, 调用 log_query(user_id, query_text); 它会生成一条包含当前时间戳、用户 ID (已登录用户名或匿名访客 ID) 和查询文本的 JSON 记录, 然后以追加模式写入 query.log 文件, 每行一个 JSON。

读取历史查询 (read_history): 调用时会打开 query.log, 按行读取所有记录并倒序遍历; 收集属于指定 user_id 的最近 top_n 次查询文本, 构成列表后返回; 如果日志文件不存在, 则返回空列表。

暴露接口 (/history): 提供一个 GET 路由, 取当前会话用户 ID, 调用 read_history 拿到其最近 10 条查询, 并以 JSON 数组形式返回。

通过这种方式, 前端可以在用户界面上展示个人的搜索历史, 增强使用体验

```

1 # 记录查询日志
2 # -----
3 def log_query(user_id, query_text):
4     rec = {
5         "timestamp": datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
6         "user":      user_id,
7         "query":     query_text
8     }
9     with open(QUERY_LOG, "a", encoding="utf-8") as f:
10         f.write(json.dumps(rec, ensure_ascii=False) + "\n")
11
12 # -----
13 # 读取历史查询
14 # -----

```

```

15 def read_history(user_id, top_n=10):
16     history_list = []
17     try:
18         with open(QUERY_LOG, "r", encoding="utf-8") as f:
19             lines = f.readlines()
20             for line in reversed(lines):
21                 rec = json.loads(line.strip())
22                 if rec.get("user") == user_id:
23                     history_list.append(rec.get("query"))
24                     if len(history_list) >= top_n:
25                         break
26     except FileNotFoundError:
27         pass
28     return history_list

```

我们可以在搜索框下面看到用户的搜索历史记录。



Figure 1: Enter Caption

2.3.7 网页快照

在爬虫阶段, `save_html_to_file` 会依据站点名在 `html_snapshots/<site_name>/` 目录下为每个 URL 生成一个唯一的文件名(把 URL 路径当成基名, 不断自增避免重名), 然后把原始 HTML 以 UTF-8 写入本地文件, 并返回这个相对路径 `snap`。随后, 脚本会把这个 `snap` 路径存入每条 `doc_meta` 里的 `snapshot` 字段, 以便前端知道如何引用。这个路由 (`/snapshot/<path:path>`) 用于将爬虫保存的本地 HTML 快照作为静态文件返回给客户端。它调用 Flask 的 `send_from_directory`, 以 `SNAPSHOT_DIR` 目录为根, 根据 URL 中的 `path` 参数定位具体文件, 并将其原样返回给浏览器, 从而让用户在搜索结果页面点击“查看快照”时, 能够直接看到与线上页面一致的离线备份内容

```
# 保存页面快照
snap = save_html_to_file(site_name, url, resp.text)
title = soup.title.string.strip() if soup.title else ""
content = soup.get_text(strip=True, separator=" ")
results.append({
    "source": site_name,
    "type": "html",
    "url": url,
    "title": title,
    "content": content,
    "snapshot": snap
})
```

```
1 @app.route("/snapshot/<path:path>")
2 def view_snapshot(path):
3     return send_from_directory(
4         os.path.abspath(SNAPSHOT_DIR),
5         path
6     )
```

在前端中，我们对搜索结果返回的所有的网页都添加了一个“快照按钮”按钮，点击“快照”，就会直接看到与线上页面的一致内容。为了避免最终的文件过大，我们只保存了 html 代码的文本信息，没有下载对应的背景图片。



Figure 2: 返回的结果带有“快照”



Figure 3: 点击快照之后显示具体页面

2.4 个性化搜索

个性化搜索通过两方面实现：在 `search_vsm` 函数中接收用户偏好关键词列表 `boost_terms`，在打分阶段对标题或摘要中命中偏好词的文档额外加分；同时在搜索结果输出前调用 `reorder_by_click`，读取用户历史点击日志，将点击过的文档优先排列，从而结合用户注册时设置的兴趣标签和实际点击行为，动态调整每次检索的结果顺序。

```

1 # ----- 在 search_vsm 中加入偏好关键词提升 -----
2 def search_vsm(query, candidate_docs=None, top_k=50, boost_terms=None,
3               boost_score=1.0):
4     # ... (TF-IDF + 余弦计算省略) ...
5
6     # 如果有 boost_terms (用户偏好关键词)，对包含这些词的文档加额外分数
7     if boost_terms:
8         for doc_id in list(score.keys()):
9             text = (doc_meta[doc_id]["title"] + " " +
10                    doc_meta[doc_id]["snippet"]).lower()
11             # 只要偏好词之一出现，就对该文档得分 + boost_score
12             for bt in boost_terms:
13                 if bt.lower() in text:
14                     score[doc_id] += boost_score
15                     break
16
17     # 最终排序，返回 top_k
18     ranked = sorted(score.items(), key=lambda x: x[1], reverse=True)
19     return [doc_id for doc_id, _ in ranked[:top_k]]
20
21 # ----- 记录点击日志 -----
22 @app.route("/click", methods=["POST"])
23 def click():
24     data = request.get_json()
25     doc_id = data.get("doc_id")
26     user_id = session.get("username", session.get("visitor_id"))
27     rec = {"user": user_id, "doc_id": doc_id}
28     with open(CLICK_LOG, "a", encoding="utf-8") as f:
29         f.write(json.dumps(rec, ensure_ascii=False) + "\n")
30     return "", 204
31
32 # ----- 基于点击历史的重排序 -----
33 def read_clicks(user_id):
34     clicked = set()
35     try:
36         with open(CLICK_LOG, "r", encoding="utf-8") as f:
37             for line in f:
38                 rec = json.loads(line.strip())
39                 if rec.get("user") == user_id:
40                     clicked.add(rec.get("doc_id"))
41     except FileNotFoundError:
42         pass

```

```
41     return clicked
42
43 def reorder_by_click(user_id, doc_ids):
44     clicked_set = read_clicks(user_id)
45     # 先把用户点击过的排前面，再加上其它没点击过的
46     clicked_list = [d for d in doc_ids if d in clicked_set]
47     others       = [d for d in doc_ids if d not in clicked_set]
48     return clicked_list + others
49
50 # ----- 在 search_page 里集成两者 -----
51 if raw_q:
52     # ... 日志、是否文档检索等略 ...
53     matched = search_vsm(raw_q, None, boost_terms=user_prefs, boost_score=1.0)
54     matched = reorder_by_click(user_id, matched)
55     # matched 就是经过“偏好提升 + 点击重排”后的最终文档 ID 列表
```



注册新账号

用户名:

密码:

确认密码:

偏好关键词 (用逗号分隔, 可留空):

填写后搜索时会优先展示包含这些词的结果。

[注册](#)

已有账号? [去登录](#)

Figure 4: 注册时可以添加偏好



Figure 5: 第一次搜索，没有点击任何页面



Figure 6: 点击后优先排列

2.5 web 页面

整个系统采用 Flask 后端 + 静态 HTML/CSS/JavaScript 前端的经典架构：后端负责加载倒排索引和文档元数据，提供 RESTful 路由（如 /search、/suggest、/history、/click、/snapshot）完成检索、联想、历史与点击日志等功能；前端用 Jinja2 模板渲染初始页面、通过 AJAX 异步调用 /suggest 和 /history 获取联想词和历史记录，并在用户输入、切换检索类型和点击结果时动态更新 UI；CSS 保证整体简洁一致的视觉风格，JavaScript 负责输入防抖、下拉联想、表单预处理与点击日志上报，将用户交互与后端服务无缝衔接。

```

1 <body>
2   <div class="container">
3     <!-- 顶部导航: Logo + 登录/注册/欢迎/登出 -->
4     <div class="nav">
5       <div class="logo">校内搜索</div>
6       <div class="user-info">
7         {% if username %}
8           欢迎, {{ username }}
9           <a href="{{ url_for('logout') }}">登出</a>
10          {% else %}
11            <a href="{{ url_for('login') }}">登录</a> /
12            <a href="{{ url_for('register') }}">注册</a>
13          {% endif %}
14        </div>
15      </div>
16
17      <!-- 搜索区域 -->
18      <div class="search-area">
19        <div class="search-type">
20          <label><input type="radio" name="search_type" value="normal" checked>
21            普通检索</label>
22          <label><input type="radio" name="search_type" value="phrase"> 短语检索</label>
23          <label><input type="radio" name="search_type" value="wildcard">
24            通配检索</label>
25          <label><input type="radio" name="search_type" value="document">
26            文档检索</label>
27        </div>
28
29        <div class="search-box">
30          <form id="search-form" method="post" action="{{ url_for('search_page') }}">
31            <button type="submit"><span class="icon"> </span></button>
32            <input type="text" id="search-input" name="q"
33              placeholder="在校内搜索..." autocomplete="off"
34              value="{{ query|e }}" autocomplete="off">
35          </form>
36          <div class="placeholder-note" id="placeholder-note">
37            直接输入关键词，支持中文或英文。
38          </div>
39
40          <!-- 联想下拉列表 -->

```

```

38     <ul id="suggest-list"></ul>
39 </div>
40
41 <!-- 历史搜索 -->
42 <div class="history">
43     <strong>历史搜索: </strong>
44     <span id="history-list">加载中... </span>
45 </div>
46 </div>
47
48 <!-- 搜索结果信息 -->
49 {% if results %}
50     <div class="results-info">
51         约 {{ results|length }} 条结果, 用时 0.12 秒
52     </div>
53 {% endif %}
54
55 <!-- 搜索结果列表 -->
56 <ul class="results-list">
57     {% for item in results %}
58     <li class="result-item">
59         {% if item.type == "document" %}
60             <span style="color: #d14; font-weight: bold; margin-right:
61                 6px;">【文档】 </span>
62             <a href="#" onclick="recordClick('{{ item.doc_id }}', '{{ item.url }}')"
63                 class="result-title">{{ item.title }}</a>
64             <div class="result-url">{{ item.url }}</div>
65             <div class="result-snippet">{{ item.snippet|safe }}</div>
66             {% if item.type == "html" and item.snapshot %}
67                 <div><a href="{{ url_for('view_snapshot', path=item.snapshot) }}"
68                     target="_blank" class="snapshot-link">查看快照</a></div>
69             {% endif %}
70         </li>
71     {% endfor %}
72 </ul>

```

2.6 个性化推荐

个性化推荐存在两类, 一个是搜索上的联想关联, 一个是内容分析后的推荐。我选择搜索上的联想关联。这段代码实现了一个基于前缀匹配的搜索联想功能:

- 它从请求参数 prefix 中读取用户已输入的前缀,
- 在全局倒排索引的所有词条里筛选出以该前缀开头的词,
- 根据每个词在多少篇文档中出现过 (df[term]) 降序, 再按词典序排序,
- 最后返回匹配度最高的前 5 个词, 供前端在输入框下拉联想时显示。

```
1 def suggest():
2     prefix = request.args.get("prefix", "").strip().lower()
3     if not prefix:
4         return json.dumps([], ensure_ascii=False)
5
6     # 找出所有以 prefix 开头的 term
7     # inverted_index 是全局词典: term -> [(doc_id, positions), ...]
8     candidates = []
9     for term in inverted_index.keys():
10        if term.startswith(prefix):
11            # df[term] 是 term 在多少篇文档出现过
12            candidates.append((term, df.get(term, 0)))
13    if not candidates:
14        return json.dumps([], ensure_ascii=False)
15
16    # 按 df 降序排序, 再按 term 字典序, 最多取前 5 个
17    candidates.sort(key=lambda x: (-x[1], x[0]))
18    top_terms = [t for t, _ in candidates[:5]]
19
20    return json.dumps(top_terms, ensure_ascii=False)
```



3 总结与思考

总体上说, 整个 web 搜索引擎的构建过程还是比较复杂的, 比较耗费时间的, 尽管花了很多时间, 最后实现的效果还比较一般啊。

但是收获还是很多的。在完整构建这一套搜索引擎的过程中, 实现了断点续爬、HTML 快照保存和文档链接提取, 使我掌握了爬虫的容错与性能优化; 自己动手构建了支持中英混合分词、位置记录的倒排索引, 体会了 TF-IDF 向量模型与余弦相似度的原理与实现; 在此基础上又扩展了短语检索、通配符检索与个性化排序功能, 让我认识到“精确匹配”与“用户画像结合”带来的灵活性与挑战; 最后, 将整个系统用 Flask 串联成完整的 Web 服务, 更加熟悉了前后端交互与会话管理。